

Hacking pyrsistent

David N. & Karl K.

University of Minnesota

16 August 2024

Background

David is interested in using `pyrsistent` as a more flexible data validation tool, with the added benefit of immutability to avoid modification by external functions, but needs something slightly different.

- ▶ `CheckedPMap` requires all fields to be the same datatype, but arbitrary correctly-typed fields can be added dynamically.
- ▶ `PRecord` supports multiple datatypes, but requires all fields to be specified in the definition of the child class.

After discussion, the decided-upon requirement was a collection that supported multiple datatypes as well as dynamic entry, while maintaining immutability.

Hacking - Q: Why Slideshows and Code Don't Mix

```

def set(self, key,
↳ original_value):
    field =
↳ self._destination_cls._precord_fields.get(key)
    if field:
        if self._factory_fields is
↳ None or field in
↳ self._factory_fields:
            try:
                if
↳ is_field_ignore_extra_complaint(PRecord,
↳ field,
↳ self._ignore_extra):
                    value =
↳ field.factory(original_value,
↳ ignore_extra=self._ignore_extra)
            else:
                value =
↳ field.factory(original_value)
        except
↳ InvariantException
↳ as e:
            ↳ self._invariant_error_codes
            ↳ +=
            ↳ e.invariant_errors

            ↳ self._missing_fields
            ↳ +=
            ↳ e.missing_fields
            return self
    else:
        value = original_value

        ↳ check_type(self._destination_cls,

```

```

def set(self, key,
↳ original_value):
    value = original_value
    field =
↳ self._destination_cls._precord_fields.get(key)
    if field:
        if self._factory_fields is
↳ None or field in
↳ self._factory_fields:
            try:
                value =
↳ field.factory(original_value)
            except
↳ InvariantException
↳ as e:
                ↳ self._invariant_error_codes
                ↳ +=
                ↳ e.invariant_errors

                ↳ self._missing_fields
                ↳ +=
                ↳ e.missing_fields
            return self
        ↳ check_type(self._destination_cls,
↳ field, key, value)

        is_ok, error_code =
↳ field.invariant(value)
        if not is_ok:
            ↳ self._invariant_error_codes.append(error_code)

    return
↳ super(_PRecordExtEvolver,

```

We hacked it properly!

Testing and Packaging

We walked through the original unit tests and discussed which were appropriate, what we should add, etc.. While not exhaustive, tests verified our desired functionality.

Code is available at:

<https://github.com/UMNLibraries/pyrsistent-ext.git>

Thanks!